Department of Electronics & Electrical Communication Engineering
Indian Institute of Technology Kharagpur

Multimedia Systems And Applications (EC60104)

Assignment 2: Delta Modulation and DPCM

Group No. : 1
Members: Adipta Halder (21EC37023)
Mayukh Shubhra Saha (21EC37009)
Abhradeep De (21EC37022)

NOTE : Due to an initial confusion, the assignment on DCT was initially submitted as Assignment 2 on 16 Feb 2026. After clarification, this report on DM and DPCM is being resubmitted. This is our final submission for Assignment 2.

PROBLEM STATEMENT

Part 1

1. (a) Write a computer program to implement a delta modulation-based encoder and decoder.

(b) Apply it to a monochrome image from the image archive. Use the following values :

(i) $\varsigma = \pm 5$, (ii) $\varsigma = \pm 10$, (iii) $\varsigma = \pm 20$.Observe the reconstructed image in each case and comment on the granular noise and slope overload observed, if any.

(c) In each case, compute the PSNR of the reconstructed image.

Part 2

2. (a) Apply the fourth-order prediction equation on a monochrome image with the coefficient. values 0.1, 0.4, 0.1, 0.4, and obtain the predicted image. Consider all pixels beyond the boundary to have the nearest boundary pixel value.

(b) Obtain the error image in (a) and plot the error image histogram.

(c) Determine the error image variance, model its distribution as Laplacian and design Lloyd-Max quantizers for (i) 2, (ii) 4, and (iii) 8 levels, based on the Lloyd-Max quantization table for unit variance Laplacian.

(d) Write a computer program to implement a DPCM encoder and decoder.

(e) For each of the quantizers designed in (c), obtain the reconstructed images and compute their PSNR values.

FIGURE 0: Original Image - lena.png (256*256 monochrome)

Theory

Lossy Predictive Coding - Foundational Principle

Lossy predictive coding provides the theoretical foundation for both Delta Modulation and DPCM. In any predictive coding scheme, the current pixel value $s(n)$ is predicted from previously reconstructed neighboring pixels, yielding a predicted value $\hat{s}(n)$. The prediction error $e(n) = s(n) - \hat{s}(n)$ is then quantized rather than transmitted as-is. The key design constraint is that both the encoder and decoder must use identical predictors, operating on reconstructed (not original) pixel values to prevent error accumulation. This is why the encoder contains an internal feedback loop that mirrors the decoder's reconstruction path exactly.

Delta Modulation (DM)

Delta Modulation is the simplest form of lossy predictive coding. It uses a first-order predictor - predicting the current pixel solely from the immediately preceding reconstructed pixel - and a two-level quantizer. The prediction equation is:

$$\hat{s}(n) = \alpha \cdot \dot{s}(n-1)$$

where α is the prediction coefficient ($\alpha \leq 1$). The quantized error is:

$$\dot{e}(n) = +\zeta \text{ if } e(n) \geq 0, \text{ and } -\zeta \text{ otherwise}$$

This encodes each pixel at exactly 1 bit/pixel, making it extremely compact but prone to two characteristic distortions:

- **Granular Noise:** When the actual pixel variation is much smaller than the step size ζ, the reconstructed signal oscillates back and forth around the true value, producing a noisy, grainy appearance in uniform regions.
- **Slope Overload:** When the actual pixel variation is much larger than the step size ζ (e.g., at sharp edges), the reconstructed signal cannot track the true signal fast enough, leading to blurred edges. Small ζ favors slope overload; large ζ favors granular noise.

Differential Pulse Code Modulation (DPCM)

DPCM generalizes Delta Modulation by using a higher-order predictor and a multi-level quantizer. The prediction is a weighted combination of multiple causal neighbors:

$$\hat{s}(n_1, n_2) = a_1 \cdot s(n_1-1, n_2-1) + a_2 \cdot s(n_1-1, n_2) + a_3 \cdot s(n_1-1, n_2+1) + a_4 \cdot s(n_1, n_2-1)$$

The prediction error is quantized using a **Lloyd-Max quantizer**, which is optimal in the Mean Squared Error sense for a given statistical distribution. Empirical observation shows that prediction errors in natural images follow a **Laplacian distribution**, which is sharply peaked at zero and falls off exponentially. The Lloyd-Max quantizer exploits this by concentrating reconstruction levels densely near zero. For images where the error variance $\sigma^2 \neq 1$, the unit-variance Lloyd-Max tables are scaled by the measured standard deviation σ.

Solution

NOTE : All relevant scripts, input and output files are present in the GitHub repository, under the directory **"Assignment_2"**. Inside this folder, there is also a detailed **README.md** file which explains the file structure, file contents and commands to run the scripts.

GitHub Repo Link : https://github.com/mayukhss262/Multimedia_Assignments

Problem 1 - Delta Modulation

- **Step 1 - Image Acquisition and Preparation:** The standard 256 × 256 grayscale Lena image is loaded using PIL's Image.open() and converted to single-channel grayscale using .convert('L'). It is then cast into a NumPy float32 array to enable precise arithmetic

without integer overflow or truncation during intermediate computations.

- **Step 2 - Delta Modulation Encoder Logic (Row-by-Row Processing):** The algorithm processes the image one row at a time, mimicking the 1D causal structure of Delta Modulation. For each row, the initial reconstructed value is set to 128.0 (mid-gray), representing a neutral starting prediction in the absence of any prior context. For each subsequent pixel in the row, the prediction is formed as $\alpha \times$ (previous reconstructed value). The prediction error is computed and its sign determines the quantized error: $+\zeta$ if non-negative, $-\zeta$ otherwise. The reconstructed pixel is clipped to [0, 255].
- **Step 3 - Parameterization over ζ values:** The algorithm is executed three separate times with $\zeta \in \{5, 10, 20\}$ while α is held constant at 1.0. This systematic sweep allows direct comparison of reconstruction quality across the three regimes of quantization step size, isolating the effect of ζ on the two characteristic noise types.

- **Step 4 - Quantitative Evaluation via PSNR:** For each ζ, the Peak Signal-to-Noise Ratio is computed as $\text{PSNR} = 10 \cdot \log_{10}(255^2 / \text{MSE})$, where MSE is the mean squared pixel-wise error between the original and reconstructed arrays. PSNR serves as the objective quality metric, complementing the subjective visual observations.

- **Step 5 - Output Persistence:** Each reconstructed image is saved as a PNG file named **reconstructed_zeta_{ζ}.png**. All PSNR values and qualitative observations are written to **part_1_dm_results.txt**.

Problem 2 - DPCM

- **Step 1 - Image Loading:** Identical to Problem 1 - PIL load, grayscale conversion, and float32 NumPy cast.

- **Step 2 - Open-Loop Prediction (Part a):** The fourth-order prediction equation from the course notes (equation 7.3) is applied using coefficients $a_1 = 0.4$ (left), $a_2 = 0.4$ (top), $a_3 = 0.1$ (top-left), $a_4 = 0.1$ (top-right). Note: "open-loop" here means the prediction is computed directly from the *original* pixel values, not reconstructed ones - this is used to generate the error image and histogram for analysis purposes only (Parts a and b). Boundary pixels outside the image are handled by clamping the row/column indices to the nearest valid boundary value.

- **Step 3 - Error Image Generation and Histogram (Part b):** The error image $e(r, c) = \text{original}(r, c) - \text{predicted}(r, c)$ is computed from the open-loop pass. Since errors span the range $[-255, +255]$, the error array is linearly scaled into [0, 255] via the formula:

error_display = clip((error + 255) / 2, 0, 255) for visualization. A frequency histogram of the raw error values (binned over the range [−100, 100]) is plotted and saved using Matplotlib, visually confirming the Laplacian shape of the error distribution.

- **Step 4 - Error Variance Computation and Lloyd-Max Quantizer Scaling (Part c):** The variance σ^2 and standard deviation σ of the error image are computed using np.var() and np.sqrt(). The unit-variance Lloyd-Max quantizer tables for 2, 4, and 8 levels (sourced from standard literature for the Laplacian distribution) are then scaled by σ: thresholds_scaled = unit_thresholds × σ and reconstruction_scaled = unit_reconstruction × σ. This produces three separate quantizers tuned to the actual statistical spread of the prediction error.

- **Step 5 - Closed-Loop DPCM Encoder/Decoder (Parts d and e):** Unlike the open-loop pass, the DPCM encoder uses previously *reconstructed* pixel values (not original values) to form each prediction. This faithfully mirrors the hardware architecture and prevents drift between encoder and decoder. For each pixel, the prediction is formed from its reconstructed causal neighbors, the error is computed and passed through the appropriate Lloyd-Max quantizer, and the reconstructed pixel is formed as prediction + quantized_error, clipped to [0, 255]. This closed-loop is executed three times - once per quantizer (2, 4, and 8 levels).

- **Step 6 - PSNR Computation and Output Persistence:** PSNR is computed for each quantizer setting using the same formula as Problem 1. Reconstructed images are saved as **dpcm_reconstructed_{levels}_levels.png**, and all numerical results (variance, thresholds, reconstruction levels, and PSNR) are written to **part_2_dpcm_results.txt**.

Code Explanation

• delta_modulation.py

Function: delta_modulation(img_array, zeta, alpha=1.0)

This is the core computational engine of the Delta Modulation implementation. It accepts the image as a float32 NumPy array, the quantization step size **zeta**, and the prediction coefficient **alpha** (defaulting to 1.0 in compliance with the assignment specification).

The function initializes **reconstructed** as a zero-filled float32 array of the same dimensions as the input, which will be populated pixel by pixel during encoding.

The outer loop **for r in range(rows)** iterates over each row of the image. At the start of every row, **prev_reconstructed** is reset to 128.0. This is a deliberate design choice: since Delta Modulation is a 1D causal scheme processing each row independently, there is no "memory" carried across rows. Initializing to mid-gray (128.0) provides a statistically neutral starting point that minimizes the average initial error.

Inside the inner loop **for c in range(cols)**, the following sequence of operations mirrors the theoretical block diagram of the DM encoder exactly:

- **Prediction:** prediction = alpha * prev_reconstructed - This implements $\hat{s}(n) = \alpha \cdot \dot{s}(n-1)$.
- **Error Computation:** error = img_array[r, c] - prediction - This computes $e(n) = s(n) - \hat{s}(n)$.
- **Quantization:** The two-level quantizer is implemented via a simple conditional: if **error >= 0**, then **quantized_error = zeta**; otherwise **quantized_error = -zeta**. This directly implements the DM quantization rule $\dot{e}(n) \in \{+\zeta, -\zeta\}$.
- **Reconstruction:** recon_pixel = prediction + quantized_error - This implements $\dot{s}(n) = \dot{e}(n) + \hat{s}(n)$, the decoder equation.
- **Bounding:** An explicit clamp to [0, 255] prevents the reconstructed value from violating the valid 8-bit pixel range - essential when edge oscillations push the signal to extremes.
- **State Update:** prev_reconstructed = recon_pixel - This closes the feedback loop. The reconstructed (not original) pixel is fed back as the input to the predictor for the next step, correctly mirroring the decoder behavior.

The function returns the fully populated **reconstructed** array.

Function: main()

This function serves as the orchestration layer - it handles I/O, parameterization, and result aggregation.

- **Image Loading:** Image.open(img_path).convert('L') loads the image and forces single-channel grayscale conversion. np.array(img, dtype=np.float32) converts it to a floating-point NumPy array for arithmetic precision.
- **Parameter Sweep:** The list **zetas = [5, 10, 20]** and the scalar **alpha = 1.0** define the full experimental configuration. The loop for zeta in zetas drives three independent invocations of delta_modulation().
- **PSNR Computation:** Inside the loop, np.mean((img_array - recon_array) ** 2) computes the element-wise MSE. The PSNR formula 10 * math.log10((255.0 ** 2) / mse) converts this to decibels. The use of math.log10 rather than np.log10 is adequate here since the input is a scalar, not an array.

- **Image Saving:** recon_array.astype(np.uint8) converts the float32 reconstructed array back to 8-bit integers before saving with PIL. The direct cast (without rounding) means values like 37.5 are truncated to 37 - a minor but acceptable approximation.
- **Results Writing:** A sequential write to a .txt file records qualitative observations (hardcoded based on theoretical understanding) followed by the measured PSNR values, creating a self-contained solution document.

- # dpcm.py

Function: get_pixel(img_array, r, c)

A utility boundary-handling function. It accepts an array and target coordinates **r**, **c**, clamps both to the valid index range using max(0, min(r, rows-1)) and max(0, min(c, cols-1)), and returns the boundary-nearest pixel value. This implements the assignment's explicit instruction that "pixels beyond the boundary have the nearest boundary pixel value" - equivalent to zero-order hold / replication padding at all four edges.

Function: predict_open_loop(img_array)

This function computes the *open-loop* predicted image and the corresponding error image from the *original* pixel values - used exclusively for Parts (a), (b), and (c) of Problem 2 (statistical analysis). It is NOT the closed-loop DPCM encoder.

For every pixel at position (r, c), it retrieves four causal neighbors via **get_pixel()**:

- **left** = pixel at (r, c−1)
- **top** = pixel at (r−1, c)
- **top_left** = pixel at (r−1, c−1)
- **top_right** = pixel at (r−1, c+1)

The prediction formula is: pred_val = 0.4 * left + 0.4 * top + 0.1 * top_left + 0.1 * top_right, which directly encodes the coefficients $a_1 = 0.4$, $a_2 = 0.4$, $a_3 = 0.1$, $a_4 = 0.1$ from equation (7.3) in the course notes. The error error[r, c] = img_array[r, c] - pred_val is stored in a separate float32 array.

The function returns both **predicted** and **error** arrays, feeding Part (b)'s histogram and Part (c)'s variance calculation.

Function: get_lloyd_max_table_unit_variance_laplacian(levels)

This function acts as a lookup table for pre-computed Lloyd-Max quantizer parameters derived for a unit-variance Laplacian distribution. It is parameterized by **levels** $\in \{2, 4, 8\}$.

For each supported level count, it returns two NumPy arrays:

- **thresholds**: The decision boundaries that partition the input domain into quantization regions. For M levels, there are M−1 thresholds.
- **reconstruction**: The M optimal reconstruction values (output levels), one per quantization region, minimizing MSE under the Laplacian distribution assumption.

These values are sourced from standard quantization theory literature:

- **2 levels:** thresholds = [0.0], reconstruction = [-0.7071, 0.7071] - symmetric around zero, as expected for a zero-mean Laplacian.
- **4 levels:** thresholds = [-1.102, 0.0, 1.102], reconstruction = [-1.834, -0.395, 0.395, 1.834] - the inner reconstruction levels are clustered closer to zero, reflecting the heavy central peak of the Laplacian.
- **8 levels:** Eight reconstruction levels with seven thresholds, exhibiting increasingly fine granularity near zero.

This function has no dependencies on other functions in the script; it is a pure parameter provider.

Function: quantize(value, thresholds, reconstruction)

A scalar quantizer implementing the Lloyd-Max decision rule. It iterates through **thresholds** in order and returns the corresponding **reconstruction** level for the first threshold that the input **value** falls below. If the value exceeds all thresholds, the last (rightmost) reconstruction level is returned. This correctly implements the piecewise constant mapping of a scalar quantizer.

This function is called pixel-by-pixel inside the **dpcm()** function for each prediction error value.

Function: dpcm(img_array, thresholds, reconstruction)

This is the closed-loop DPCM encoder-decoder, the central function of Problem 2. It is architecturally distinct from **predict_open_loop()** in one critical way: all predictions are formed from previously *reconstructed* pixel values, not original ones - faithfully implementing the feedback loop described in Fig. 7.1 of the course notes.

The function maintains **reconstructed_img** as the accumulating output array. For each pixel at (r, c):

- **Causal Neighbor Retrieval (with fallback):** Rather than using the **get_pixel()** utility, the function uses inline conditionals with a fallback default of **128.0** for pixels at the image boundary (top row and left column). This choice of 128.0 (mid-gray) as the boundary initialization is consistent with the DM encoder's initialization strategy and ensures no information leakage from future pixels or invalid memory accesses.

    - left = get_pixel(reconstructed_img, r, c-1) if c > 0 else 128.0
    - top = get_pixel(reconstructed_img, r-1, c) if r > 0 else 128.0
    - top_left = get_pixel(reconstructed_img, r-1, c-1) if (r > 0 and c > 0) else 128.0
    - top_right = get_pixel(reconstructed_img, r-1, c+1) if (r > 0 and c < cols-1) else 128.0
- **Prediction:** pred_val = 0.4 * left + 0.4 * top + 0.1 * top_left + 0.1 * top_right - identical coefficients to the open-loop predictor.

- **Error and Quantization:** err = img_array[r, c] - pred_val computes the true error against the *original* pixel. **quantize(err, thresholds, reconstruction)** maps this to the nearest Lloyd-Max reconstruction level.

- **Reconstruction and Clamping:** recon_pixel = pred_val + err_q reconstructs the pixel; the result is clamped to [0, 255].

- **State Update:** reconstructed_img[r, c] = recon_pixel stores the reconstructed value, which will be used as a causal neighbor for subsequent pixels. This is the feedback path.

The function returns the complete **reconstructed_img** array.

Function: calculate_psnr(original, reconstructed)

A standalone PSNR utility, structurally identical to the PSNR logic in delta_modulation.py. It returns float('inf') if MSE = 0 (perfect reconstruction), and the standard PSNR formula otherwise. Keeping it as a named function (rather than inline code) improves modularity and readability.

Function: main()

The orchestration function for Problem 2, executing the full analysis pipeline:

1. **Image Loading** - same pattern as Problem 1.
2. **Open-loop prediction call** - predict_open_loop(img_array) returns **predicted** and **error** arrays for Part (a).

3. **Error Image Visualization** - scales error to [0, 255] and saves as dpcm_error_image.png.
4. **Histogram Generation** - plt.hist(error.flatten(), bins=100, range=(-100, 100)) flattens the 2D error array and plots the frequency distribution. The range [−100, 100] is chosen to capture the dominant central region of the Laplacian without being distorted by rare extreme outliers.
5. **Variance and Standard Deviation Computation** - np.var(error) and np.sqrt(variance) produce the scaling parameters for Lloyd-Max table lookup.
6. **Quantizer Design Loop** - iterates over **levels** ∈ {2, 4, 8}: calls get_lloyd_max_table_unit_variance_laplacian(levels) to get unit tables, scales by **std_dev**, logs the resulting threshold and reconstruction arrays, calls **dpcm()** with the scaled parameters, computes PSNR, and saves the reconstructed image.
7. **Results Export** - all numerical results are joined and written to a .txt file.

Outputs and Observations

Problem 1 - Delta Modulation

Results:

ζ Value	PSNR (dB)
ζ = 5	17.08
ζ = 10	19.54
ζ = 20	21.48

Observations:

- **ζ = 5 (Small Step Size):** Slope overload is the dominant artifact. The step size is too small to track the steep intensity gradients at edges in the Lena image. The reconstructed signal lags significantly behind rapid intensity changes, causing prominent blurring at boundaries. Uniform smooth regions may be relatively preserved. PSNR is lowest at 17.08 dB.

- **ζ = 10 (Medium Step Size):** An intermediate result is observed. Neither slope overload nor granular noise is highly dominant; both are present to a moderate degree. The

reconstruction trades off between the two distortion types, yielding a visually acceptable but still imperfect result. PSNR improves to 19.54 dB.

- **ζ = 20 (Large Step Size):** Granular noise dominates. In smooth, uniformly colored regions of the Lena image (e.g., the background and skin tones), the reconstruction oscillates around the true value with amplitude ±20, producing a visually grainy, textured appearance in regions that should be smooth. Edges are tracked better but the noise floor is high. PSNR is highest at 21.48 dB - note that numerically higher PSNR does not always translate to better perceptual quality; granular noise distributes distortion more uniformly, which the MSE metric rewards over the spatially concentrated blur of slope overload.

Reconstructed Images:

FIGURE 1: Reconstructed Image - ζ = 5 (reconstructed_zeta_5.png)

FIGURE 2: Reconstructed Image - ζ = 10 (reconstructed_zeta_10.png)

FIGURE 3: Reconstructed Image - ζ = 20 (reconstructed_zeta_20.png)

Problem 2 - DPCM

Results :

Parameter	Value

Error Variance (σ^2)	219.08
Error Standard Deviation (σ)	14.80

Scaled Lloyd-Max Quantizer Parameters:

Levels	Thresholds	Reconstruction Levels	PSNR (dB)
2	[0.0]	[−10.47, 10.47]	22.17
4	[−16.31, 0.0, 16.31]	[−27.15, −5.85, 5.85, 27.15]	27.27
8	[−35.22, −18.54, −7.89, 0.0, 7.89, 18.54, 35.22]	[−45.69, −24.76, −12.33, −3.45, 3.45, 12.33, 24.76, 45.69]	31.67

Observations:

- **Error Image:** The error image should visually appear as a low-contrast, near-gray image with slight brightening at sharp edges (where prediction error is high) and near-uniform mid-gray in smooth regions (where prediction error is close to zero and maps to 128 after scaling).

- **Error Histogram:** The histogram is expected to show a sharply peaked, symmetric, bell-shaped distribution centered tightly at zero, with rapidly decaying tails - the hallmark Laplacian profile. This statistically justifies the use of Lloyd-Max quantizers designed for Laplacian distributions. The distribution is narrower and more peaked than a Gaussian of the same variance, confirming that most prediction errors in the Lena image are small, with occasional large errors at edges.

- **2-Level Quantizer (PSNR = 22.17 dB):** With only two reconstruction levels (±10.47), the quantized error is a coarse binary approximation. Significant blocking and contouring artifacts are expected in the reconstructed image, particularly in smooth gradient regions. Despite this, PSNR of 22.17 dB already outperforms Delta Modulation at $\zeta = 5$, demonstrating the power of even a simple DPCM predictor over pure DM.

- **4-Level Quantizer (PSNR = 27.27 dB):** A marked improvement of approximately 5 dB over the 2-level case. The reconstruction is visually much closer to the original, with fine

details in the Lena image (hair, feathers) beginning to resolve. Contouring is reduced significantly.

- **8-Level Quantizer (PSNR = 31.67 dB):** The highest quality reconstruction in this study, approximately 4 dB above the 4-level case. The image is visually close to the original, with most artifacts confined to areas of very sharp edges. This result demonstrates that 3 bits/pixel of DPCM coding achieves substantially higher quality than 1 bit/pixel Delta Modulation, at the cost of a larger quantizer alphabet.

Reconstructed Images:

FIGURE 5:Error Image (dpcm_error_image.png)

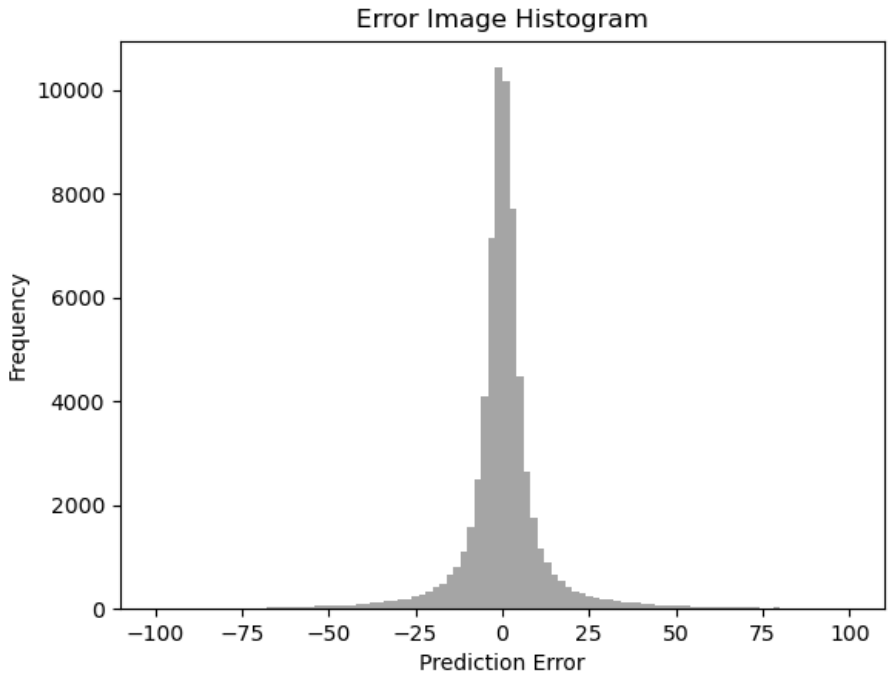

FIGURE 6: Error Image Histogram - Laplacian Distribution Shape (dpcm_error_histogram.png)

FIGURE 7: DPCM Reconstructed Image - 2 Levels (dpcm_reconstructed_2_levels.png)

FIGURE 8: DPCM Reconstructed Image - 4 Levels (dpcm_reconstructed_4_levels.png)

FIGURE 9: DPCM Reconstructed Image - 8 Levels (dpcm_reconstructed_8_levels.png)

Conclusion

This assignment provided a comprehensive practical exploration of two foundational lossy predictive image compression schemes — Delta Modulation and Differential Pulse Code Modulation — implemented and evaluated on the standard 256 × 256 grayscale Lena image. The Delta Modulation experiments confirmed, both theoretically and empirically, the fundamental trade-off governed by the quantization step size ζ: small values of ζ introduce slope overload, manifesting as blurred edges where the reconstructed signal cannot track rapid intensity transitions, while large values of ζ produce granular noise, causing spurious oscillations in smooth, uniform regions of the image. This inherent tension arises from the extreme constraint of encoding each pixel at only 1 bit/pixel using a single-parameter two-level quantizer, and it cannot be resolved without adapting ζ dynamically to local image statistics — the motivation behind Adaptive Delta Modulation. The measured PSNR values of 17.08 dB, 19.54 dB, and 21.48 dB for $\zeta = 5$, 10, and 20 respectively quantified this trade-off objectively, while also highlighting the well-known limitation of PSNR as a perceptual quality metric: a numerically higher PSNR at $\zeta = 20$ does not necessarily imply a more visually acceptable image, since granular noise, though energetically distributed, is perceptually disruptive in smooth regions. The DPCM experiments demonstrated the substantial gains achievable by moving to a higher-order predictor and a multi-level Lloyd-Max quantizer. The fourth-order predictor effectively exploited the spatial correlation among neighboring pixels, reducing the error variance from the full image variance to a much smaller value of 219.08 ($\sigma \approx 14.80$), with the resulting error distribution closely following the expected Laplacian profile — sharply peaked at zero with rapidly decaying tails — thereby justifying the application of Lloyd-Max quantization theory. The progressive improvement in PSNR from 22.17 dB (2 levels, 1 bit/pixel) to 27.27 dB (4 levels, 2 bits/pixel) and finally 31.67 dB (8 levels, 3 bits/pixel) starkly illustrates that each additional bit of quantization precision yields significant gains in reconstruction fidelity, with the 8-level DPCM result outperforming the best Delta Modulation result by over 10 dB — a decisive demonstration of the advantage of combining a strong predictor with an optimally designed, statistically-matched non-uniform quantizer. Collectively, these results reinforce the core theoretical principle that effective lossy compression is achieved not merely by quantizing the raw signal, but by first removing predictable redundancy through intelligent prediction and then quantizing only the residual uncertainty — the smaller and more concentrated this residual, the more efficiently it can be encoded.